

# FX1 Sports (FX1)

---

# SMART CONTRACT

---

## Security Audit

**Performed on Contracts:**

FX1SportsToken.sol

MD5 Hash: f560c31a8bc1ace2cf1c5c39edec6cbb

Platform

ETH

hashlock.com.au

SEPTEMBER 2023

# Table of Contents

|                                   |    |
|-----------------------------------|----|
| Executive Summary                 | 4  |
| Project Context                   | 4  |
| Audit scope                       | 7  |
| Security Rating                   | 8  |
| Standardised Checks               | 9  |
| Function List                     | 11 |
| Intended Smart Contract Functions | 14 |
| Code Quality                      | 15 |
| Audit Resources                   | 15 |
| Dependencies                      | 15 |
| Severity Definitions              | 16 |
| Audit Findings                    | 16 |
| High                              | 17 |
| Medium                            | 19 |
| Low                               | 25 |
| Gas                               | 25 |
| Centralisation                    | 26 |
| Conclusion                        | 27 |
| Our Methodology                   | 28 |
| Disclaimers                       | 30 |
| About Hashlock                    | 31 |

## CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

# Executive Summary

The FX1 team partnered with Hashlock to conduct a security audit of their new FX1 Sports Token Contract. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

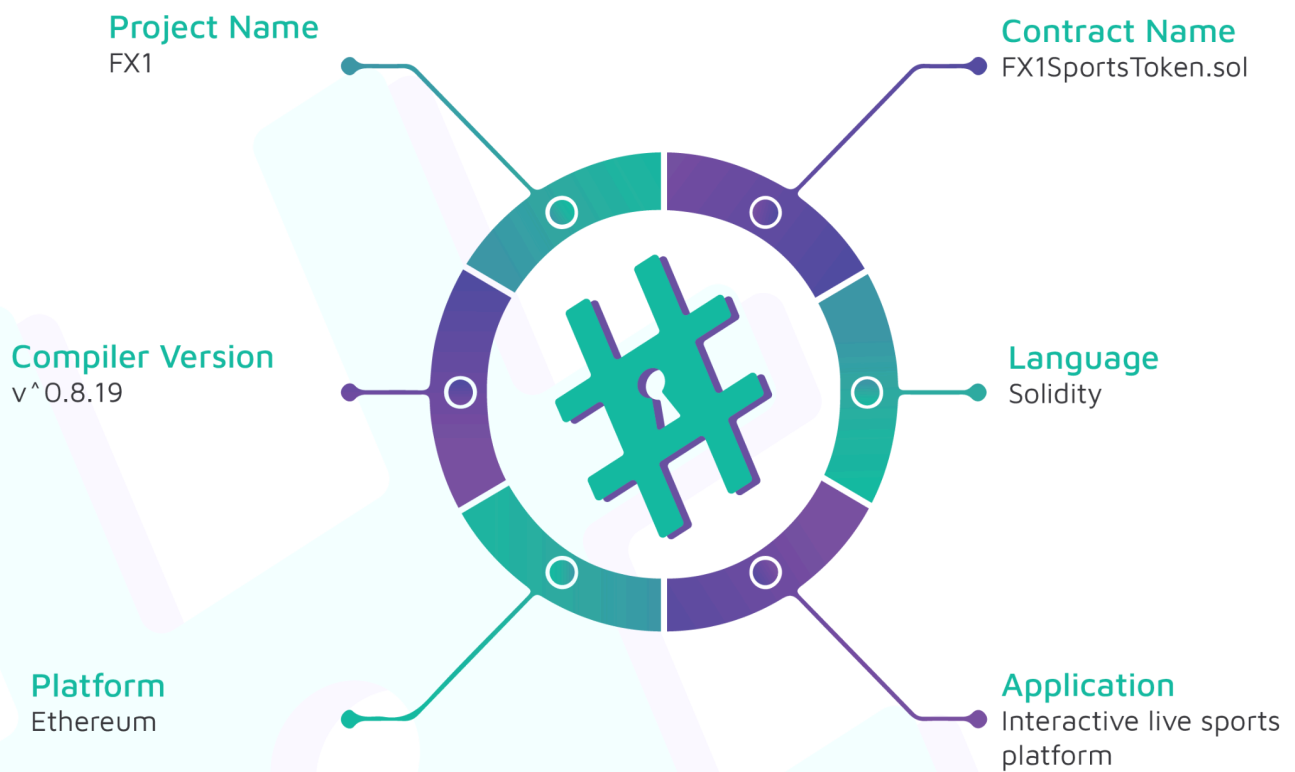
## Project Context

The FX1 team is on a mission to evolve and enhance the overall experience of watching sports online for Gen Z's and young Millennial communities. FX1 will grow viewership by enabling them to connect and engage socially while watching online sports with their friends, while also having the ability to earn income, prizes, and clout.

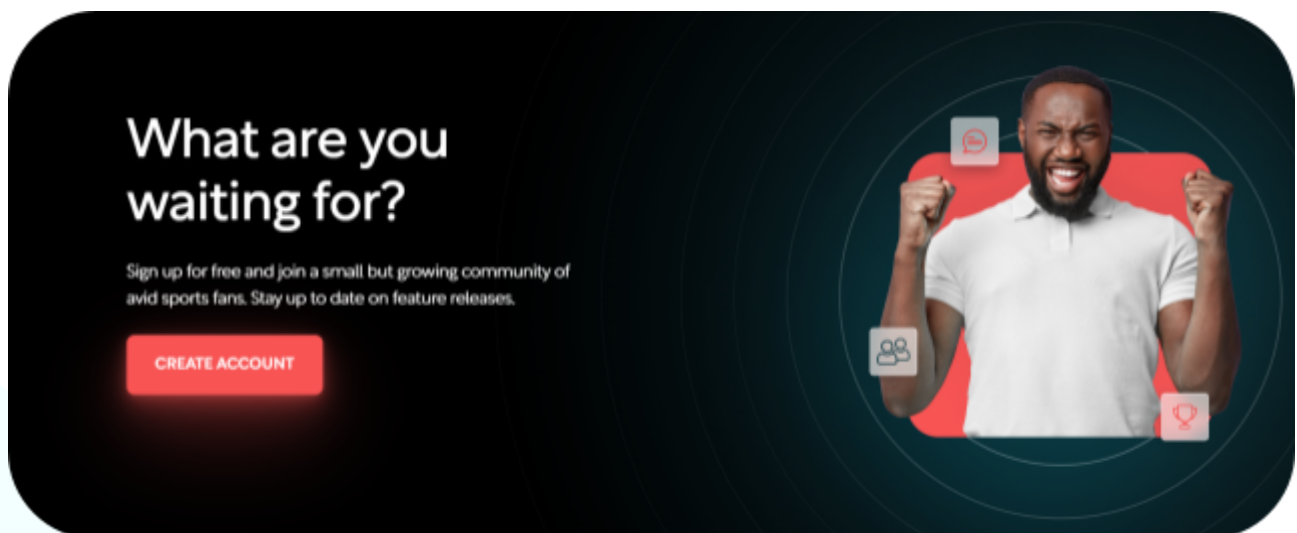
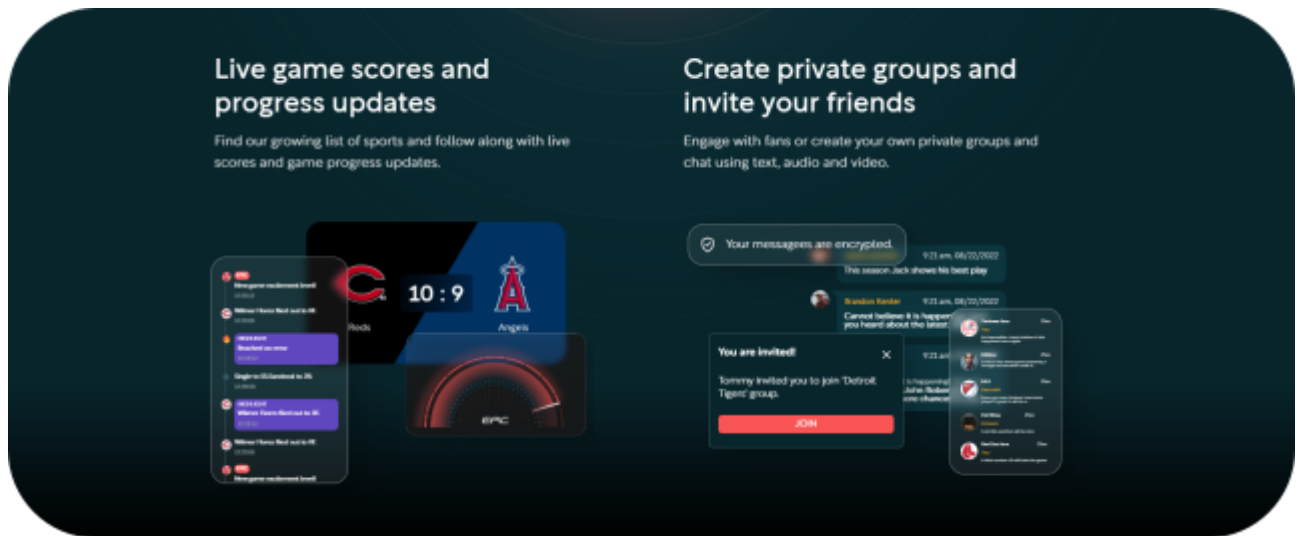
**Project Name:** FX1

**Compiler Version:** ^0.8.19

**Website:** <https://fx1.io/>

**Logo:****Visualised Context:**

## Project Visuals:



## Audit scope

We at Hashlock audited the solidity code within the FX1 Project, the scope of works included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line by line analysis and were supported by software assisted testing.

|                      |                                                                                               |
|----------------------|-----------------------------------------------------------------------------------------------|
| <b>Description</b>   | <b>Project Review and Security Analysis Report for FX1 Smart Contracts and other factors.</b> |
| <b>Platform</b>      | <b>Ethereum / Solidity</b>                                                                    |
| <b>Audit Date</b>    | <b>Sept, 2023</b>                                                                             |
| <b>Contract Name</b> | FX1SportsToken.sol                                                                            |
| <b>MD5 Hash</b>      | f560c31a8bc1ace2cf1c5c39edec6cbb                                                              |

# Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts. We have identified some significant vulnerabilities that need to be addressed before deployment.

**Not Secure****Vulnerable****Secure****Hashlocked**

*The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.*

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit overview section. General overview is presented in the Function list section and all identified issues can be found in the Audit overview section.

All vulnerabilities initially identified have now been resolved and acknowledged.

## Hashlock found:

1 High severity vulnerabilities

5 Medium severity vulnerabilities

0 Low severity vulnerabilities

0 Gas Findings

**Caution:** Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.



## Standardised Checks

| Main Category              | Subcategory                                    | Result   |
|----------------------------|------------------------------------------------|----------|
| <b>General Code Checks</b> | Solidity/compiler version stated               | Passed   |
|                            | Consistent pragma version across each contract | Passed   |
|                            | Outdated Solidity Version                      | Passed   |
|                            | Overflow/underflow                             | Passed   |
|                            | Correct checks, effects, interaction order     | Reviewed |
|                            | Lack of check on input parameters              | Reviewed |
|                            | Function input parameters check bypass         | Passed   |
|                            | Correct Access control                         | Reviewed |
|                            | Built in emergency features                    | Passed   |
|                            | Correct event logs                             | Passed   |
|                            | Human/contract checks bypass                   | Passed   |
|                            | Random number generation/use vulnerability     | Passed   |
|                            | Fallback function misuse                       | Passed   |
|                            | Race condition                                 | Passed   |
|                            | Logical vulnerability                          | Reviewed |
|                            | Features claimed                               | Reviewed |
|                            | Other programming issues                       | Reviewed |
| <b>Code Specification</b>  | Correctly declared function visibility         | Passed   |
|                            | Correctly declared variable storage location   | Passed   |
|                            | Use keywords/functions to be deprecated        | Passed   |
|                            | Unused code                                    | Reviewed |
| <b>Gas Optimization</b>    | "Out of Gas" Issue                             | Reviewed |
|                            | High consumption 'for/while' loop              | Passed   |
|                            | High consumption 'storage' storage             | Passed   |

|                        |                                       |        |
|------------------------|---------------------------------------|--------|
|                        | Assert() misuse                       | Passed |
| <b>Tokenomics Risk</b> | The maximum limit for mintage not set | Passed |
|                        | "Short Address" Attack                | Passed |
|                        | "Double Spend" Attack                 | Passed |

**Initial Audit Result: VULNERABLE**

**Revised Audit Result: PASSED**

# Function List

## FX1SportsToken.sol

### Functions

|    | Functions          | Type               | Modifiers         | Conclusion |
|----|--------------------|--------------------|-------------------|------------|
| 1  | constructor        | N/a                | initializer/write | H-02       |
| 2  | name               | external/<br>view  | no modifiers      | No Issues  |
| 3  | symbol             | external/<br>view  | no modifiers      | No Issues  |
| 4  | decimals           | external/<br>view  | no modifiers      | No Issues  |
| 5  | totalSupply        | external/<br>view  | no modifiers      | No Issues  |
| 6  | balanceOf          | external/<br>view  | no modifiers      | No issues  |
| 7  | transfer           | external/<br>write | no modifiers      | No issues  |
| 8  | allowance          | external/<br>view  | no modifiers      | No issues  |
| 9  | approve            | external/<br>write | no modifiers      | No issues  |
| 10 | transferFrom       | external/<br>write | no modifiers      | No issues  |
| 11 | updateDexRouter    | external/<br>write | onlyOwner         | No issues  |
| 12 | updatePair         | external/<br>write | onlyOwner         | No issues  |
| 13 | setWhiteListPeriod | external/<br>write | onlyOwner         | No issues  |
| 14 | multiSender        | external/<br>write | onlyOwner         | No issues  |
| 15 | setBots            | external/<br>write | onlyOwner         | No issues  |

|           |                                   |                    |              |                   |
|-----------|-----------------------------------|--------------------|--------------|-------------------|
| <b>16</b> | delBot                            | external/<br>write | onlyOwner    | No issues         |
| <b>17</b> | setLaunchBegin                    | external/<br>write | onlyOwner    | No issues         |
| <b>18</b> | updateWhitelists                  | external/<br>write | onlyOwner    | No issues         |
| <b>19</b> | excludeWalletsFromMax<br>Transfer | external/<br>write | onlyOwner    | No issues         |
| <b>20</b> | excludeWalletsFromMax<br>Wallets  | external/<br>write | onlyOwner    | No issues         |
| <b>21</b> | excludeWalletsFromFees            | external/<br>write | onlyOwner    | No issues         |
| <b>22</b> | setMaxTransferAmount              | external/<br>write | onlyOwner    | No issues         |
| <b>23</b> | setMaxWalletAmount                | external/<br>write | onlyOwner    | [M-04]            |
| <b>24</b> | setMarketingTaxWallet             | external/<br>write | onlyOwner    | No issues         |
| <b>25</b> | updateBuyFeeRate                  | external/<br>write | onlyOwner    | No issues         |
| <b>26</b> | updateSellFeeRate                 | external/<br>write | onlyOwner    | No issues         |
| <b>27</b> | setSwapThreshold                  | external/<br>write | onlyOwner    | No issues         |
| <b>28</b> | _transfer                         | internal/<br>write | no modifiers | [M-03]            |
| <b>29</b> | _swapBack                         | internal/<br>write | no modifiers | [H-01],<br>[M-01] |
| <b>30</b> | _taxonSellTransfer                | internal/<br>write | no modifiers | [M-02]            |
| <b>31</b> | _taxonBuyTransfer                 | internal/<br>write | no modifiers | [M-02]            |
| <b>32</b> | _basicTransfer                    | internal/<br>write | no modifiers | No issues         |
| <b>33</b> | _getSellFeeRate                   | internal/v<br>iew  | no modifiers | No issues         |

|           |                |                    |              |           |
|-----------|----------------|--------------------|--------------|-----------|
| <b>34</b> | _getBuyFeeRate | internal/<br>view  | no modifiers | No issues |
| <b>35</b> | _transferETH   | internal/<br>write | no modifiers | No issues |
| <b>36</b> | _approve       | private/w<br>rite  | no modifiers | No issues |

# Intended Smart Contract Functions

| Claimed Behaviour                                                                                                                                                                                                                                                                                                    | Actual Behaviour                                                                                                                            |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| <p><b>File 1 FX1SportsToken.sol</b></p> <ul style="list-style-type: none"><li>• Owner predominantly has control over the setters and critical state changing functions.</li><li>• Interacts with an AMM for token swapping when required.</li><li>• Standard ERC20 behaviour to support the FX1 ecosystem.</li></ul> | <p><b>The contract achieves mostly the expected functionality however, some edge cases exist which may cause the contract to break.</b></p> |

## Code Quality

This audit scope involves the solidity smart contracts of the FX1 project, as outlined in the Audit Scope section. All contracts, libraries and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that would increase the likelihood of exploitation, however some refactoring is still required.

The code is very well commented and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

## Audit Resources

We were given the FX1 Protocol's smart contract code in the form of a downloadable file.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in understanding the overall architecture of the protocol.

## Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

## Severity Definitions

| Significance | Description                                                                                                                                                                                           |
|--------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| High         | High severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community. |
| Medium       | Medium level difficulties should be solved before deployment, but require edge case conditions to be met in order to cause adverse effects.                                                           |
| Low          | Low level vulnerabilities are areas that lack best practices that may cause small complications in the future.                                                                                        |
| Gas          | Gas Optimisations, issues and inefficiencies                                                                                                                                                          |



# Audit Findings

## High

### [H-01] A malicious user can cease all transactions between users by forcibly sending tokens directly to an empty liquidity pool

#### Summary

All transactions can be reverted as a result of a denial of service should the liquidity not be initialised by the deployer.

#### Vulnerability Analysis

In the constructor, the contract will create a new WETH/FX1 pair to facilitate swaps for the contract should they be required. There exists a check in the `_swapBack()` function where the contract assesses if the `accTotalAmount <= swapThreshold`:

```
function _swapBack() internal {  
    uint256 accTotalAmount = accLiquidityAmount + accMarketingAmount;  
    if (accTotalAmount <= swapThreshold) {  
        Return;  
    }  
}
```

A malicious user can bypass this check by transferring an arbitrary amount of tokens directly to the empty liquidity pool in order to trigger a swap on transfer for all users who may be using the token. Because there is no liquidity in the pool, all transactions between all users will be reverted.

#### Impact

This was rated a high in severity because of how cheap and easy it was to trigger, resulting in a denial of service for all users who may want to transfer the FX1 token. The following proof of concept outlines this scenario:

```
function testDosCondition() public {  
    // Setup scenario  
    vm.startPrank(address(admin));  
    token.transfer(address(eve), 1000 ether);  
    token.transfer(address(bob), 100 ether);  
    token.setLaunchBegin();  
    address[] memory accounts = new address[](3);  
    accounts[0] = bob;
```

```
accounts[1] = eve;
accounts[2] = alice;
token.updateWhitelists(accounts, true);
token.setSwapThreshold(1 ether);
vm.stopPrank();

// Start scenario
vm.startPrank(address(eve));
token.transfer(address(token.pair()), 100 ether);
vm.stopPrank();

vm.startPrank(address(bob));
vm.expectRevert("INSUFFICIENT_LIQUIDITY");
token.transfer(address(alice), 100 ether);
vm.stopPrank();
}
```

### Recommendation

It's recommended that the deployer immediately provides a sufficient amount of liquidity to the newly created pool in the constructor.

### Status

Resolved

## Medium

### [M-01] Transfer transactions may revert if the transaction is not processed by block.timestamp

#### Summary

Block.timestamp is hard coded as the transaction deadline when swapping with the liquidity pool which may cause a revert.

#### Vulnerability Analysis

Currently block.timestamp is hard coded as the transaction deadline when the contract facilitate a swap with the liquidity pool as it can be seen from the codeblock below:

```
dexRouter.swapExactTokensForETHSupportingFeeOnTransferTokens(  
    swapAmount,  
    0,  
    path,  
    address(this),  
    block.timestamp // @audit-issue - hardcoded block.timestamp  
);
```

It can be seen from a previous issue that this corresponds to the deadline parameter in the swapExactTokensForETHSupportingFeeOnTransferTokens().

As a result, the transaction will revert if swap transactions are not processed almost immediately. This can be an issue when users attempt to transfer tokens when there is high traffic as it may take longer than usual to execute the swap.

#### Impact

User transfer transactions may be dosed if transactions are not processed immediately.

#### Recommendation

It's recommended that this parameter is user supplied and can be set as block.timestamp if not required. If this poses an issue, uniswaps default deadline in their user interface is set to 30 minutes which may be used in this situation, but caution is advised during high traffic.

#### Status

Resolved

## [M-02] Unsafe casting of taxation may cause inaccurate fees

### Summary

Because mathematical operations are performed alongside the FIXED\_POINT (uint16) state variable which is a smaller size than its counterparts (ie. amount - uint256), this may result in integer truncation and incorrect fees.

### Vulnerability Analysis

Currently, the code for \_taxonSellTransfer, there are mathematical operations performed in order to determine the fee amounts to be taxed:

```
uint16 public immutable FIXED_POINT = 1000;

uint256 feeAmount = (_amount * marketingFeeRate) / FIXED_POINT; // @audit - unsafe
casting
uint256 liquidityFeeAmount = (_amount * liquidityFeeRate) / FIXED_POINT;
uint256 marketingFeeAmount = feeAmount - liquidityFeeAmount;
uint256 recvAmount = _amount - feeAmount;
```

There exists certain edgcase scenarios where incorrect fees may be used in transfers because of integer truncation.

### Impact

An edge case condition may exist where incorrect fees could be used when collecting taxes on transfer transactions.

### Recommendation

It's recommended that the contract implements OpenZeppelin's SafeCast.sol library to ensure no integer truncations occur.

### Status

Resolved

## [M-03] maxTransferAmount check can be bypassed by batching transactions

### Summary

A user can bypass the maxTransferAmount check by sending one transaction after another in order to send more funds than the contract intended.

### Vulnerability Analysis

The FX1 token contract implements a simple check to assert that users are not transferring funds which breach the maxTransferAmount in the \_transfer function:

```
require(
    excludedFromMaxTransfer[_sender] ||
    _amount <= maxTransferAmount + (10 * 10 ** _decimals), // @audit-issue -
    maxTransferAmount can be bypassed through batched transactions
    "Exceeds to maxTransferAmount"
);
```

A user can use batch transferring or implement a smart contract which facilitates multiple transactions in order to bypass this invariant.

### Impact

Violation of maximum transfer invariant. The unit test below outlines this scenario:

```
function testBypassMaxTransferCheck() public {
    // Setup scenario
    vm.startPrank(address(admin));
    token.setLaunchBegin();
    address[] memory accounts = new address[](3);
    accounts[0] = bob;
    accounts[1] = eve;
    accounts[2] = alice;
    token.updateWhitelists(accounts, true);
    address[] memory excludeFromMaxTransfer = new address[](1);
    excludeFromMaxTransfer[0] = alice;
    vm.stopPrank();

    vm.startPrank(address(admin));
    token.transfer(address(alice), token.maxTransferAmount() + 1000 ether);
    vm.stopPrank();

    vm.startPrank(address(alice));
    token.transfer(address(bob), token.maxTransferAmount());
    token.transfer(address(bob), 1000 ether); // Immediate extra transfer
    vm.stopPrank();
```

```

        assertEquals(token.balanceOf(address(bob)), token.maxTransferAmount() + 1000
ether);
    }
}

```

## Recommendation

It's recommended that `maxTransferAmount` and `maxWalletAmount` checks are refactored to reflect the following:

```
require(  
    excludedFromMaxTransfer[_sender] ||  
    _amount <= maxTransferAmount,  
    "Exceeds to maxTransferAmount"  
);  
  
require(  
    excludedFromMaxWallet[_recipient] ||  
    _balances[_recipient] + _amount <=  
    maxWalletAmount,  
    "Exceeds to maxWalletAmount"  
);
```

## Status

Resolved

## [M-05] Number of tokens that the contract is willing to accept from the liquidity pool is zero

### Summary

Because `amountOutMin` for `swapExactTokensForETHSupportingFeeOnTransferTokens()` is hardcoded to zero, the contract will effectively be willing to accept zero tokens in return.

### Vulnerability Analysis

The interface for the `IUniswapV2Router02.swapExactTokensForETHSupportingFeeOnTransferTokens()` looks like the following:

```
function swapExactTokensForETHSupportingFeeOnTransferTokens(
    uint amountIn,
    uint amountOutMin,
    address[] calldata path,
    address to,
    uint deadline
) external;
```

The second parameter determine the minimum amount of tokens the contract will receive back however, as it can be seen from the implementation, this value is hardcoded to zero:

```
dexRouter.swapExactTokensForETHSupportingFeeOnTransferTokens(
    swapAmount,
    0, // @audit-issue - min amount out is hardcoded to zero
    path,
    address(this),
    block.timestamp
);
```

### Impact

The contract should receive the optimal amount out from the liquidity pool however, during market turbulence, frontrunning can be utilised to ensure that the contract receives a less desirable amount.

### Recommendation

It's recommended that this is user supplied input where the user can set this to zero themselves if not required.

### Auditors Note

The FX1 team understands that this discovery could influence the benefits they receive. Nevertheless, since it neither impacts nor poses any harm to FX1 users, we have designated it as a medium-priority item with the acknowledgment of the project lead.

### Status

Documented and Analysed



## Low

No low severity issues were found at the conclusion of this audit.

## Gas

No gas optimisations were found at the conclusion of this audit.

# Centralisation

The project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

## Conclusion

After Hashlocks analysis, the FX1 project seems to have a sound and well tested code base, however our findings need to be resolved in order to achieve security. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

# Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits are to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

## **Manual Code Review:**

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

## **Vulnerability Analysis:**

Our methodologies include manual code analysis, user interface interaction, and whitebox penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

**Documenting Results:**

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this we analyse the feasibility of an attack in a live system.

**Suggested Solutions:**

We search for immediate mitigations that live deployments can take and finally we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contracts details are made public.

# Disclaimers

## Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment and functionality (performing the intended functions).

Due to the fact that the total number of test cases are unlimited, the audit makes no statements or warranties on security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bugfree status or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds, and is not in any way liable for the security of the project.

## Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee explicit security of the audited smart contracts.

## About Hashlock

Hashlock is an Australian based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3 oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

**Website:** [hashlock.com.au](https://hashlock.com.au)

**Contact:** [info@hashlock.com.au](mailto:info@hashlock.com.au)



# #Hashlock.